EEEN20282 Applied Mechanics and Industrial Robotics

PC Laboratory on Forward Kinematics

Student Name: PeaceofGod Oyewole

Student ID number: 11377069

The simulation successfully modelled a 3-degree-of-freedom (DOF) elbow manipulator using the Denavit-Hardtenberg (DH) convention). Based on the student ID number and changing the second digit, the parameters $l_1 = 9$, $l_2 = 3$, $l_3 = 0$, resulting in the following DH parameters, using the table (3.2) in the lab script: The base height $d_1 = 11$m, significantly elevates the manipulator, while the link lengths $a_2 = 4$m and $a_3 = 2$m define the reach of the arm.

1. Task 1

The robot was simulated for 5 seconds using equations (3.5), (3.6) and (3.7) in the lab script. The end-effectors' Cartesian coordinates were logged and exported to MATLAB for analysis.

a. 3-D plot with x, y and z

The 3-D plot shows the special path of the end-effector.

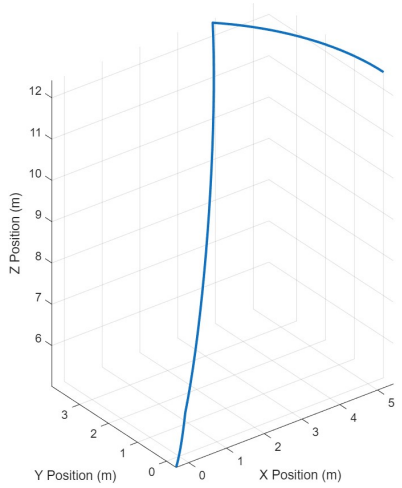


Figure 1.1: 3-D plot Trajectory

Due to the significant base height (d1 = 11m), the trajectory starts at a high elevation (z=11m). The sweep demonstrates the combined rotational influence of the base joint and the 'elbow' extension

b. 2-D plot with each coordinate vs time in a single plot.

Figure 1.2 shows the x, y and z coordinates plotted against time. The coordinates remain constant at the start ($t < 1$). As joint 2 lifts the arm, there is a peak before it levels off again at the setpoint

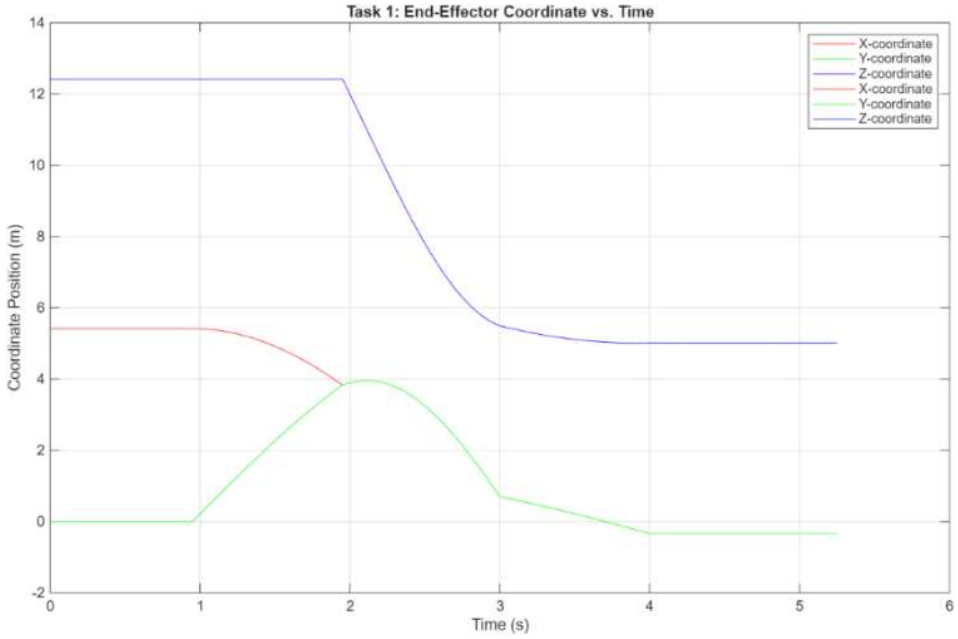


Figure 1.2: 2-D plot with each coordinate vs time

2. Task 2

Using MATLAB, calculations for all links and the end effector transformation matrices were made at three different times: 1.75s, 2.75s and 3.75s to verify the simulation model using the Homogenous Transformations equation (3.1) of the lab script. The total transformation is calculated as equation 2.1:

$$H_3^0 = H_1^0 * H_1^0 * H_3^2 \quad (2.1)$$

At time 1.75s:

$$H_1^0 = \begin{bmatrix} 0.8315 & -0.0000 & -0.5556 & 0 \\ 0.5556 & 0.0000 & 0.8315 & 0 \\ 0 & -1.0000 & 0 & 11.0000 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}, \qquad H_1\text{^}0 = \begin{bmatrix} 1 & 0 & 0 & 4 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

$$H_3^2 = \begin{bmatrix} 0.7071 & 0.7071 & 0 & 1.4142 \\ -0.7071 & 0.7071 & 0 & -1.4142 \\ 0 & 0 & 1.0000 & 0 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

$$\text{Total transformation } H_3^0 = \begin{bmatrix} 0.58791 & 0.58790 & -0.5556 & 4.5018 \\ 0.39280 & 0.3928 & 0.8315 & 3.0080 \\ 0.7071 & -0.7071 & 0.0000 & 12.4142 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

Calculated End-Effector Position: X=4.502, Y=3.008, Z=12.414

At time 2.75s:

$$H_1^0 = \begin{bmatrix} 0.7071 & -0.0000 & -0.7071 & 0 \\ 0.7071 & 0.0000 & 0.7071 & 0 \\ 0 & -1.0000 & 0 & 11.0000 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}, \quad H_2^1 = \begin{bmatrix} 0.3827 & -0.9239 & 0 & 1.5307 \\ 0.9239 & 0.3827 & 0 & 3.6955 \\ 0 & 0 & 1.0000 & 0 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix},$$

$$H_3^2 = \begin{bmatrix} 0.70711 & 0.7071 & 0 & 1.4142 \\ -0.70710 & 0.7071 & 0 & -1.4142 \\ 0 & 0 & 1.0000 & 0 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

$$\text{Total transformation } H_3^0 = \begin{bmatrix} 0.6533 & -0.2706 & -0.7071 & 2.3890 \\ 0.6533 & -0.2706 & -0.7071 & 2.3890 \\ -0.3827 & -0.9239 & 0.0000 & 6.5391 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

Calculated End-Effector Position: X=2.389, Y=2.389, Z=6.539

At time 3.75s:

$$H_1^0 = \begin{bmatrix} 0.7071 & -0.0000 & -0.7071 & 0 \\ 0.7071 & 0.0000 & 0.7071 & 0 \\ 0 & -1.0000 & 0 & 11.0000 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}, \quad H_2^1 = \begin{bmatrix} 0.0000 & -1.0000 & 0 & 0.0000 \\ 1.0000 & 0.0000 & 0 & 4.0000 \\ 0 & 0 & 1.0000 & 0 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix},$$

$$H_3^2 = \begin{bmatrix} 0.9239 & 0.3827 & 0 & 1.8478 \\ 0.3827 & 0.9239 & 0 & 0.7654 \\ 0 & 0 & 1.0000 & 0 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

$$\text{Total transformation } H_3^0 = \begin{bmatrix} -0.2706 & -0.6533 & -0.7071 & -0.5412 \\ -0.2706 & -0.6533 & 0.7071 & -0.5412 \\ -0.9239 & 0.3827 & 0.0000 & 5.1522 \\ 0 & 0 & 0 & 1.0000 \end{bmatrix}$$

Calculated End-Effector Position: X=-0.541, Y=-0.541, Z=5.152

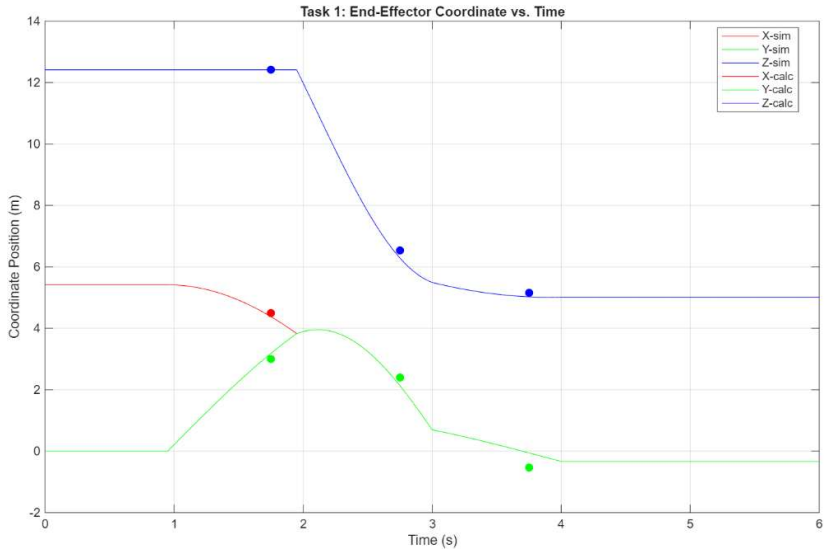


Figure 2.1: Simulation vs Calculated

The calculated points were overlaid on the simulated trajectory. The points aligned closely with the simulated lines, validating the kinematic model. Decencies raised possibly due to differences In the CoppeliaSim software model internal parameters and rounding errors in MATLAB.

3. Task 3

The UR5 6DOF robot was successfully programmed to insert a cuboid into a wall slot without disturbing the surrounding structure, by sequencing joint movements. Joint 1 on the base was rotated to face the wall (90 angle), then adjusting the wrist in joint 4 to maintain the cuboids horizontal position, and then finally extending the arm (Joint 2 and 3) for insertion. The 6_DOF configuration was very useful as it allowed independent control of the end-effector's orientation to align precisely with the slot while moving its position. While 3 DOFs are sufficient to reach a point in space, the additional 3 DOFs, the wrist joints, are required to rotate the cuboid to be perfectly parallel with the slot opening

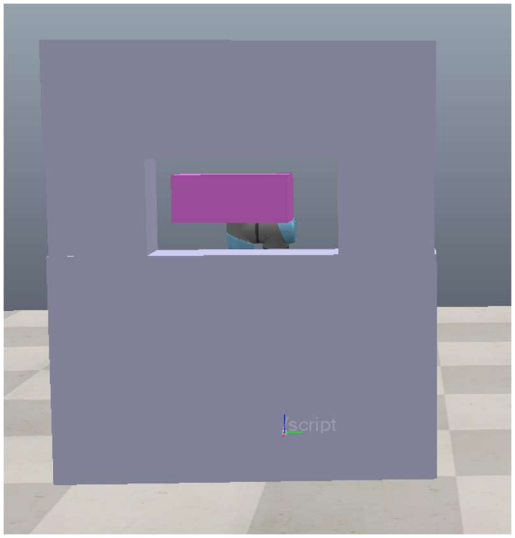


Figure A: Front View

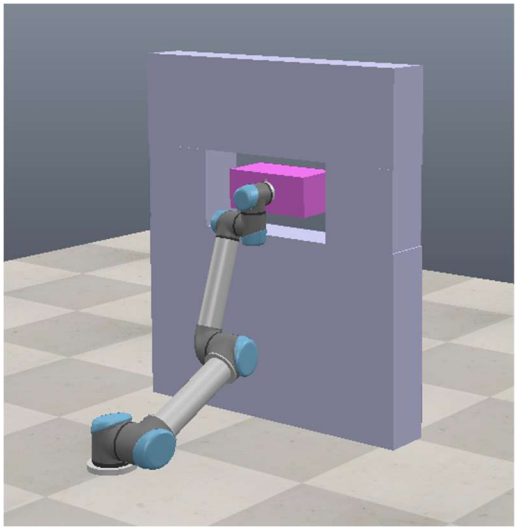


Figure B: View from an Angle

Figure 3.3: A successful end result. The cuboid is inserted entirely without

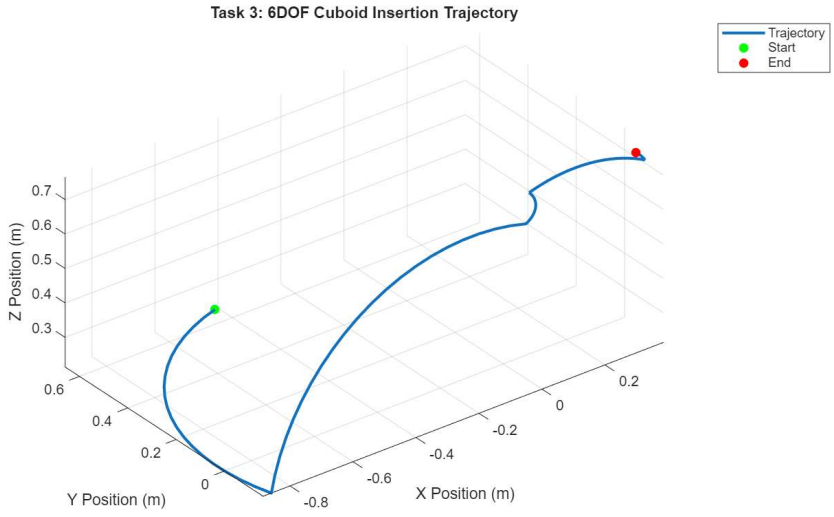


Figure 3.2: 3-D Trajectory

Figure 3.2 shows that the end-effector travels along curved arc-like paths rather than straight line towards the slot, a primary characteristic of joint movement. It shows how the robot must first orient itself (large initial curve) before extending towards the wall. The successful insertion part of the trajectory is represented by the final, smaller segment of the line that enters the wall's coordinates.

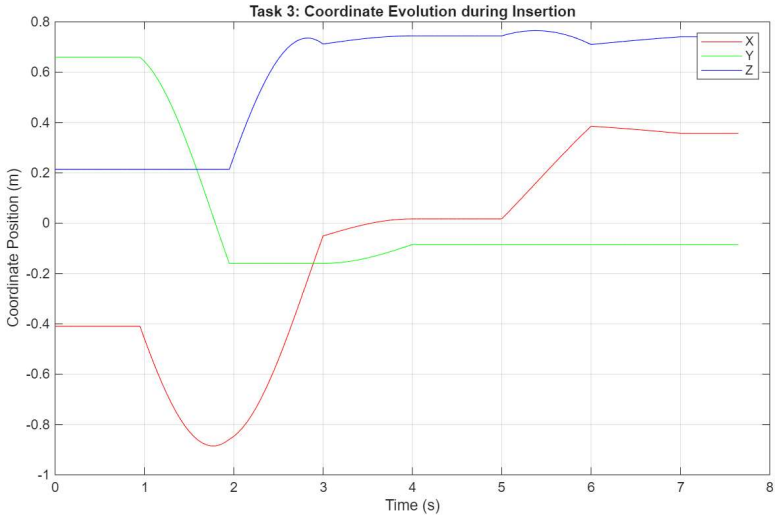


Figure3.3: 2-D plot with each coordinate vs time

Similar to task 1, figure 3.2 has flat lines at the beginning and end of the plot confirm the robot starts from a stable initial point and successfully holds its final position inside the slot. It is difficult to keep a single coordinate (e.g. Z) perfectly level while the others change, which is necessary for a horizontal insert. This attempt is represented by the dips/peaks of lines in the graph, showing the lack of coordinated cartesian path

To conclude, while trial and error and manual tuning of joint ramps can achieve the task, it is highly ineffective for complex manoeuvres. Inverse kinematics offers a solution to this where the use defines a straight-line path and the software calculates the necessary joint angles automatically.